DATING YOUR COMPUTER

a systems date facility

Walter B. Novinger

```
10 '    DATE PROGRAM
20 '    The date placed by this routine remains
30 '    intact after a warm start (RESET button)
40 '    as well as any diskette swaps (RESET state-
50 '    ment in MBASIC or ^C from CP/M).
60 '
70 '    The date is poked to locations 0050-005B
80 '
100 DATA "January   ","February   ","March ","April ","May   ","June"
110 DATA "July    ","August    ","September ","October ","November   "
120 DATA "December   "
130 OPTION BASE 1:DIM M$(12):FOR I=1 TO 12:READ M$(I):NEXT
140 PRINT:INPUT "Enter today's date as MMDDYY";DT$
150 M=VAL(LEFT$(DT$,2)):IF M<1 OR M>12 THEN 140
160 D=VAL(MID$(DT$,3,2)):IF D<1 OR D>31 THEN 140
170 Y=VAL(RIGHT$(DT$,2)):IF Y<82 OR Y>85 THEN 140
180 POKE 80,D:POKE 81,M
190 FOR I=1 TO 9:POKE 81+I,ASC(MID$(M$(M),I,1)):NEXT
200 POKE 91,Y
210 PRINT:PRINT "Page zero date buffer initialized.  If you turn"
220 PRINT "the computer off, you must re-run DATE to"
230 PRINT "re-initialize the date."
240 PRINT:PRINT "DATE removed from memory -- BASIC is ready"
250 NEW    'Deletes program leaving BASIC ready for user's input
254 '
255 '    Warning  -- Line #250 deletes the program.
256 '    Add in after the program is checked out
257 '    for keystroke errors, or first SAVE to disk.
```

Figure 1

While developing a business program for the Osborne 1, it became apparent that I would need a facility to store the current date in a place accessible to any program running under *MBASIC.* Ideally, such storage would be unmodified by CP/M, *MBASIC* or any user program. In addition, the means of accessing the date should require no machine language programming and CALLs or USRs from *BASIC* programs. My ultimate goal was to produce a program that would request the current date when *MBASIC* was booted, and a simple subroutine for accessing the date stored at boot time from within any *BASIC* module.

This article describes three elements of the Osborne date facility as I have implemented it: 1) The DATE program that requests and stores the date; 2) the GETDATE subroutine to access the date; and finally 3) AUTOST.COM modifications to make the date setting process "automatic" when *MBASIC* is booted.

DATE.BAS (Figure 1)

The first decision was to determine the location in which to store the date. After perusing the Osborne *User's Guide*, I stumbled upon a chart labeled "Memory Allocation" in the section covering assembly language (page R-243 in my manual). Here, in locations 0050H-005BH, I found what I was looking for—a block indicated as "not used." This 12-byte block was just what I needed to store dates in two formats.

After I found the buffer I needed, writing DATE was easy. I stored the date in both numeric and ASCII formats since the program that later fetches the date with GETDATE has its choice of formats without having to duplicate DATE's logic.

The code of DATE is straightforward and should cause Osborne users little difficulty. Note, however, that DATE deletes itself after storing the date. This facilitates the autoloading feature by leaving *MBASIC*

in a pristine state; exactly where the user expects it to be, ready to accept program input.

GETDATE.BAS (Figure 2)

This subroutine is used within *BASIC* programs to access the date stored by DATE. Execution of a GOSUB 200 in a program results in four variables being initialized with the date information. DAY, MONTH and YEAR contain the respective date components in numeric format. These would be useful, for instance, in placing a date in a file record (using the LSET statement to create a two-byte file record field). The string DT$ is set to the date in the form "12 May 1982"—useful for labeling listings and the like. Note that this string is returned with one leading space (like *BASIC's* STR$ function) and no trailing space.

```
1 '    GETDATE Subroutine sample
2 '    Date is stored in DT$
10 GOSUB 200
20 PRINT CHR$(26)
30 PRINT SPACE$(15);DT$
40 '  Insert your program here.
50 PRINT:PRINT "This is a test.":PRINT
60 END
200 DAY=PEEK(80):DT$=STR$(DAY)+" ":MONTH=PEEK(81):I=82
210 WHILE C$<>" " AND I<91:C$=CHR$(PEEK(I)):DT$=DT$+C$:I=I+1:WEND
220 YEAR=PEEK(91):DT$=DT$+"19"+RIGHT$(STR$(YEAR),2)
230 RETURN
```

Figure 2

AUTOST.ASM (Figure 3)

Implementation of the set-date-at-boot facility required that the AUTOST.COM program resident on the *MBASIC* diskette be modified. Fortunately, the *User's Guide* provides instructions for doing exactly this (if you can follow the rather cryptic comments). In Chapter 8, "*CP/M Revisited,*" there is a discussion of CP/M's ASM and LOAD programs; included with the discussion is a listing of the AUTOST.ASM source. The text which is stored beginning at label FILENAME is placed in the CP/M command buffer by the subroutine labeled AGAIN. When the PCHL instruction at address 0138H is executed, CP/M will use this text as a command to be executed. As you can see from the listing in Figure 3, this command will be *MBASIC* DATE. Note that this text must be followed by at least one zero-byte to terminate the command (see listing comments). *MBASIC* then loads and executes the file DATE.BAS, our DATE program. Obviously, DATE.BAS must be resident on the *MBASIC* booting diskette in Drive A.

```
AUTOST.ASM  Modifications
Copy the AUTOST.ASM program (version 1) exactly as
it appears in the Osborne User's Guide in Chapter 8
except for the following modifications.

Step 1
Change this line    LXI     B,10    ;set counter
to                  LXI     B,13    ;set counter

This sets the character counter.  The command string (MBASIC
DATE) was longer than 8 characters so LXI B,length becomes
LXI B,length+2.

Step 2
Change the command line to:

FILENAME: DB    11,'MBASIC DATE',0,0,0,0

      length of       command
       command         string

Step 3
Finally, add a few message lines.  There is room
for  maximum of 8 lines of code:

ENDMES: DB  CR,LF,LF,LF,9,'Loading MBASIC to initialize date',CR,LF,LF
        DB  9,'When requested, enter today',39,'s date',CR,LF
        DB  9,'in the format shown in the prompt',CR,LF,'$'
```

Figure 3

The second major modification involves the message that AUTOST displays after printing the Osborne logo. This message is stored beginning at label ENDMES and is displayed (in subroutine PRINT) by a BDOS routine. This routine merrily prints characters until it encounters a $; thus, the last character of the ENDMES message is the dollar sign.

When you have used *WordStar* (in non-document mode) to enter AUTOST.ASM and make the modifications described, assemble and load the resulting program. I suggest that you do this on a scratch diskette (to avoid clobbering anything useful).

The inclusion of the system date facility has made business programming on the Osborne much more professional-looking as well as easier to write. The AUSTOST facility provided with the Osborne is extremely handy; taken together with the documentation and the source code listing, the user has the beginnings of a powerful customization facility.